



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA INGENIERÍA EN SOFTWARE

TEMA:

**GA -- Práctica en clase Instalación de los entornos y primeros script
PHP y PERL**

PRESENTADO POR GRUPO C:

FIGUEROA MORALES BRYAN JAVIER

FECHA:

VIERNES, 5 DE JUNIO DEL 2025
(Semana 5)

1. Instalación y Verificación del Entorno de Desarrollo

1.1 Entorno utilizado

Se instaló la versión 8.2.12 de XAMPP Apache + MariaDB + PHP + Perl



Imagen 1 XAMPP version 8.2.12

1.2 Verificación de versiones instaladas

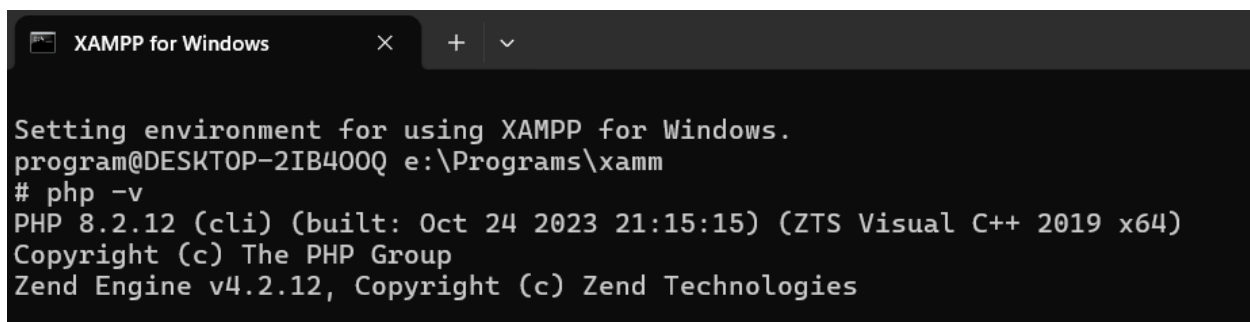


Imagen 2 PHP 8.2.12 (cli)

```
program@DESKTOP-2IB400Q e:\Programs\xamm
# perl -v

This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread
Copyright 1987-2021, Larry Wall

Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl".  If you have access to the
Internet, point your browser at http://www.perl.org/, the Perl Home Page.
```

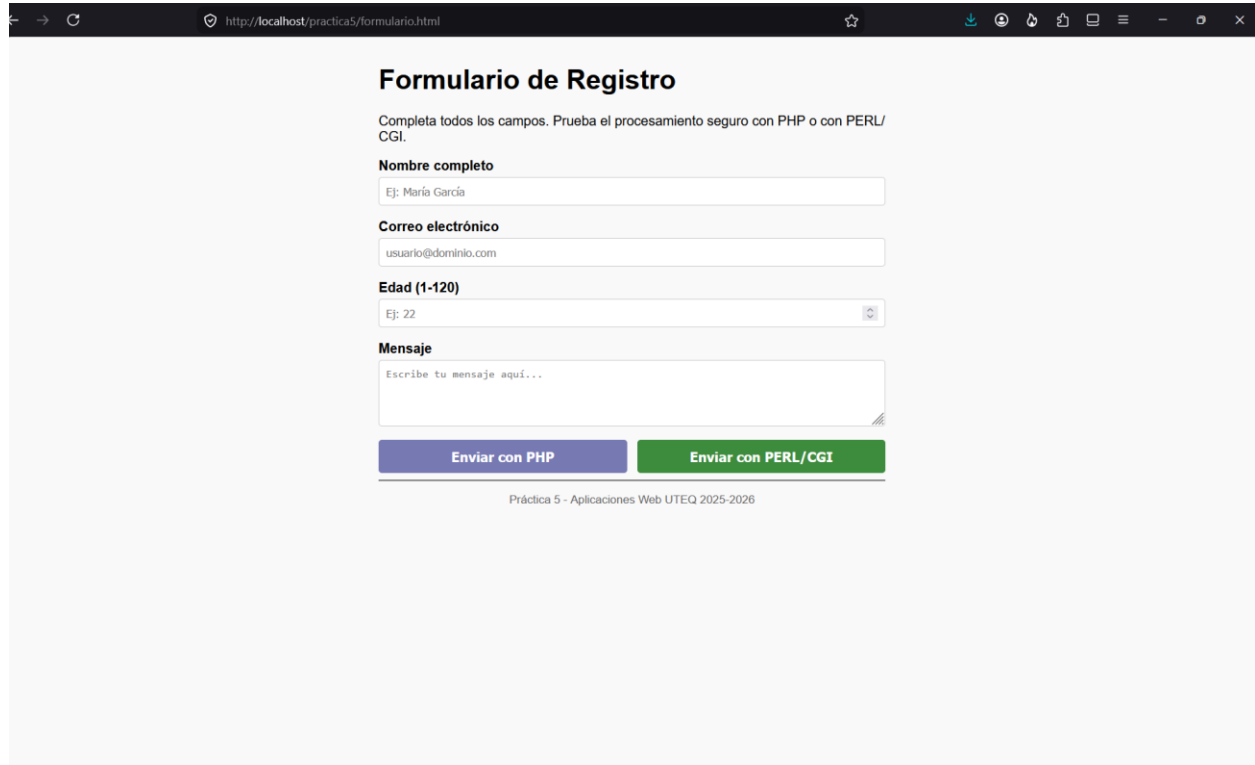
Imagen 3 PERL (v5.32.1)

2. Formulario HTML Común

2.1 Estructura del formulario

El archivo formulario.html contiene un único formulario con cuatro campos (nombre, email, edad, mensaje) y dos botones de envío: uno hacia procesar.php y otro hacia /cgi-bin/procesar.pl, usando el atributo formaction.

2.2 Visualización en el navegador



The screenshot shows a web browser window with the address bar displaying 'http://localhost/practica5/formulario.html'. The page content is a registration form titled 'Formulario de Registro'. Below the title, there is a instruction: 'Completa todos los campos. Prueba el procesamiento seguro con PHP o con PERL/CGI.' The form consists of four input fields: 'Nombre completo' (with a placeholder 'Ej: María García'), 'Correo electrónico' (with a placeholder 'usuario@dominio.com'), 'Edad (1-120)' (with a placeholder 'Ej: 22' and a spinner icon), and 'Mensaje' (with a placeholder 'Escribe tu mensaje aquí...'). At the bottom of the form, there are two buttons: 'Enviar con PHP' (purple) and 'Enviar con PERL/CGI' (green). Below the buttons, there is a footer text: 'Práctica 5 - Aplicaciones Web UTEQ 2025-2026'.

Imagen 4 Formulario.html

2.3 Estructura de archivos creada

Describir brevemente la ubicación de cada archivo:

- `htdocs/practica5/formulario.html`
- `htdocs/practica5/procesar.php`
- `cgi-bin/procesar.pl`

3. Script PHP 8.2 — procesar.php

3.1 Descripción del script

El script procesar.php verifica que el método de envío sea POST, lee y valida cada campo con `filter_input()`, aplica `htmlspecialchars()` en toda salida para prevenir XSS, y muestra un resumen de datos válidos o los mensajes de error específicos.

3.2 Funciones PHP clave utilizadas

- `filter_input()` — Lee variables de `INPUT_POST` aplicando filtros de validación o saneamiento
- `FILTER_VALIDATE_EMAIL` — Valida formato de email según RFC 5321
- `FILTER_VALIDATE_INT` — Valida entero con opciones de rango mínimo/máximo
- `htmlspecialchars()` — Convierte caracteres especiales HTML en entidades (prevención XSS)
- `declare(strict_types=1)` — Activa modo de tipos estrictos en PHP 8

3.3 Evidencia de funcionamiento

http://localhost/practica5/formulario.html

Formulario de Registro

Completa todos los campos. Prueba el procesamiento seguro con PHP o con PERL/CGI.

Nombre completo

Correo electrónico

Edad (1-120)

Mensaje

Enviar con PHP **Enviar con PERL/CGI**

Práctica 5 - Aplicaciones Web UTEQ 2025-2026

Imagen 5 Formulario con datos completos



Imagen 6 Procesar.php funcionando con datos válidos

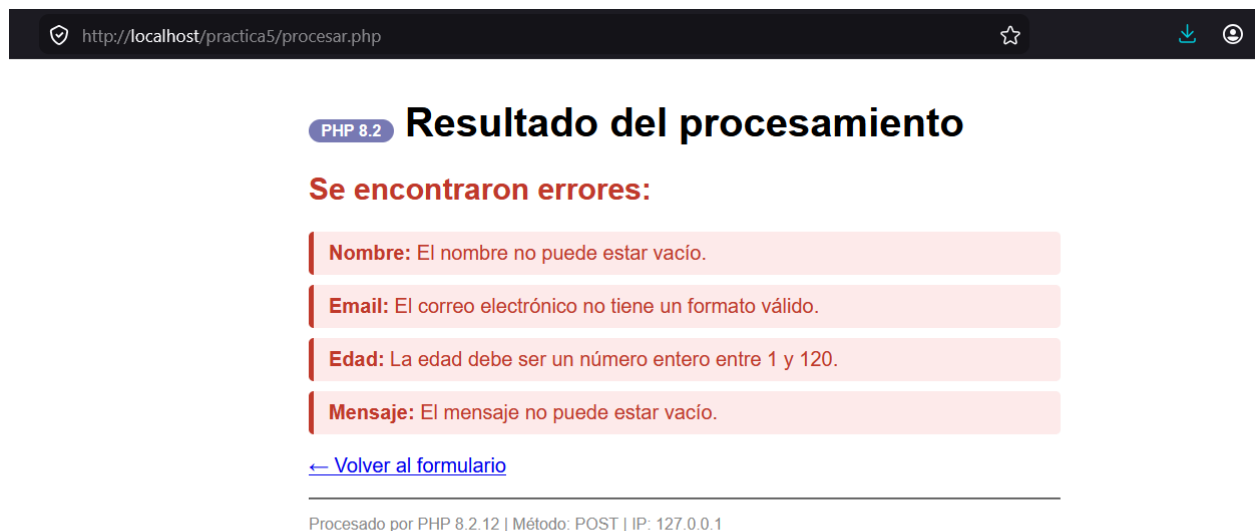


Imagen 7 Procesar.php mostrando errores de validación

4. Script PERL/CGI — procesar.pl

4.1 Descripción del script

El script procesar.pl usa los pragmas use strict y use warnings, crea un objeto CGI con CGI->new, emite obligatoriamente la cabecera HTTP como primera salida, valida cada campo y genera la respuesta HTML mediante bloques heredoc.

4.2 Elementos PERL/CGI clave utilizados

- use strict — Obliga a declarar variables con my; evita errores por nombres mal escritos
- use warnings — Activa advertencias en tiempo de ejecución
- CGI->new — Crea objeto que encapsula la petición HTTP
- \$cgi->param() — Lee parámetros del formulario (equivalente a filter_input de PHP)
- CGI::escapeHTML() — Escapa caracteres HTML especiales (equivalente a htmlspecialchars)
- Heredoc — Bloques HTML multilínea sin concatenar cadenas

4.3 Evidencia de funcionamiento

http://localhost/practica5/formulario.html

Formulario de Registro

Completa todos los campos. Prueba el procesamiento seguro con PHP o con PERL/CGI.

Nombre completo

Correo electrónico

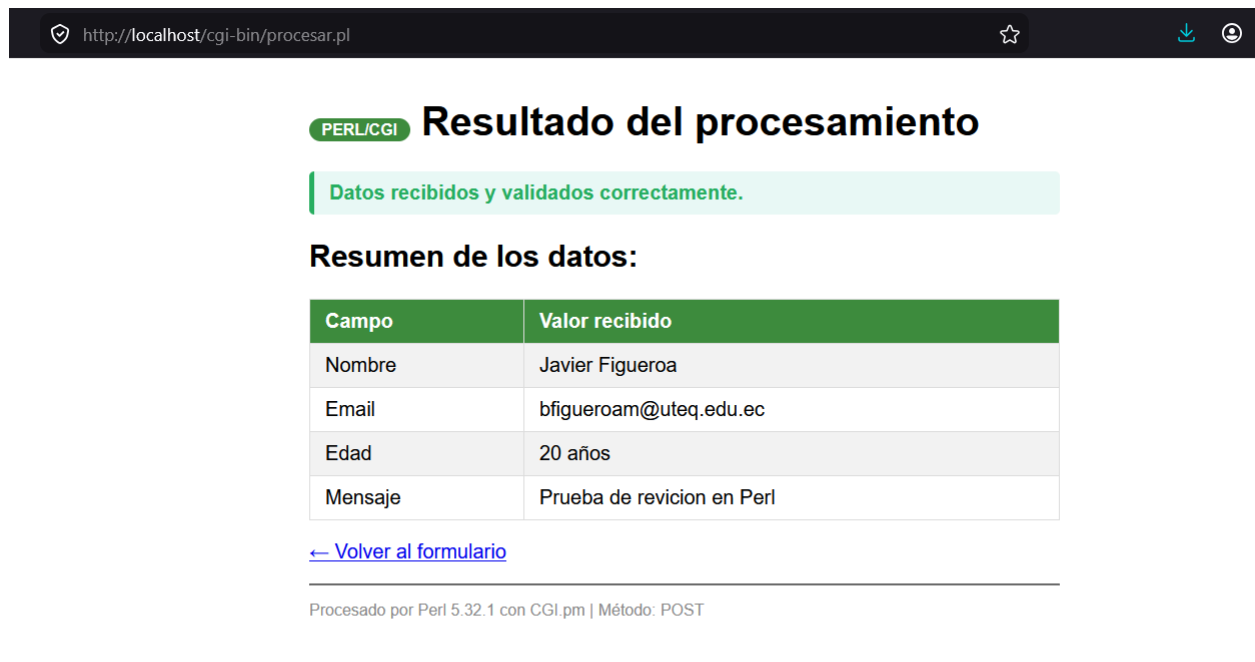
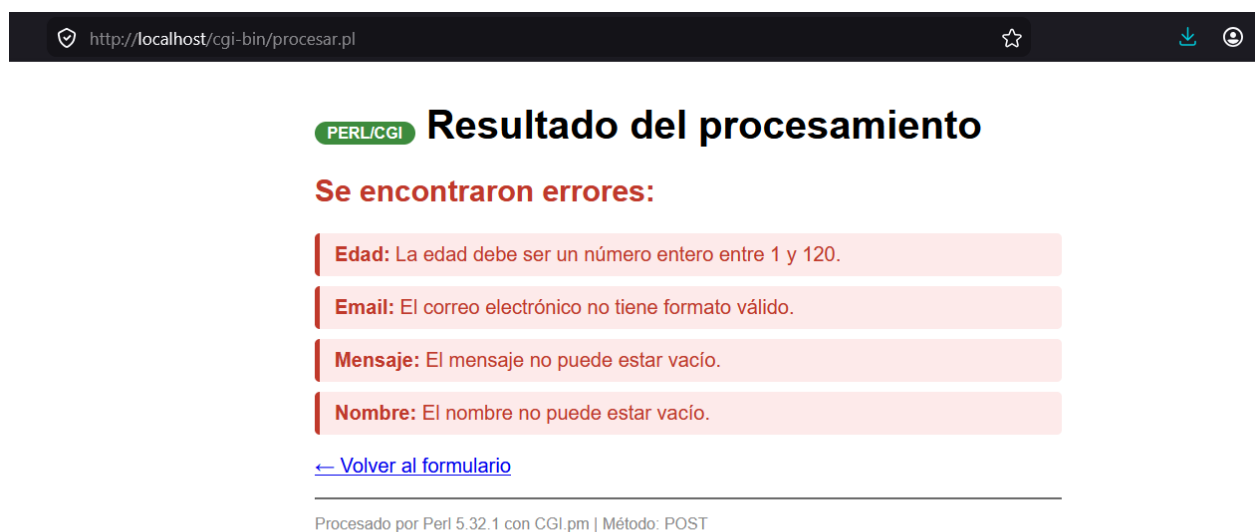
Edad (1-120)

Mensaje

Enviar con PHP **Enviar con PERL/CGI**

Práctica 5 - Aplicaciones Web UTEQ 2025-2026

Imagen 8 Formulario con datos completos

*Imagen 9 Procesar.pl funcionando con datos válidos**Imagen 10 Procesar.pl mostrando errores de validación*

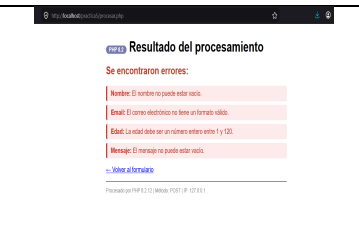
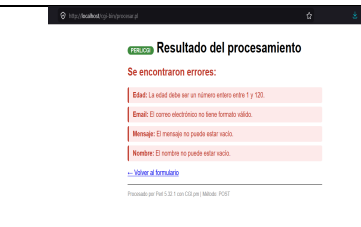
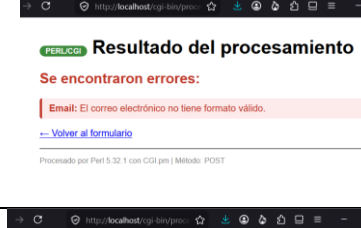
5. Evidencia de los Casos de Prueba

Se probaron los 10 casos de prueba definidos en la guía, tanto con PHP como con PERL/CGI.

#	Caso	Datos de entrada	Resultado esperado	PHP	PERL
1	Campos vacíos	Formulario sin rellenar	Error en los 4 campos	SI	SI
2	Email sin dominio	nombre=Ana, email=usuario@, edad=20, msg=Hola	Error solo en email	SI	SI
3	Email sin arroba	email=usuariodominio.com	Error solo en email	SI	SI
4	Edad negativa	edad=-5	Error: fuera de rango 1-120	SI	SI
5	Edad muy alta	edad=200	Error: fuera de rango 1-120	SI	SI
6	Edad no numérica	edad=veinte	Error: debe ser entero	SI	SI
7	XSS en nombre	nombre=<script>alert(1)</script>	Texto escapado <script>...</script>	SI	SI
8	XSS en mensaje	mensaje=	Etiqueta como texto sin ejecución	SI	SI
9	Datos válidos	Todos los campos correctos	Resumen sin errores	SI	SI
10	Nombre con caracteres especiales	nombre=María José Muñoz-López	Datos aceptados y mostrados	SI	SI

5.1 Casos de prueba 1–6 (Validaciones)

Incluir una captura por cada caso mostrando claramente el dato enviado y la respuesta del servidor, tanto en PHP como en PERL.

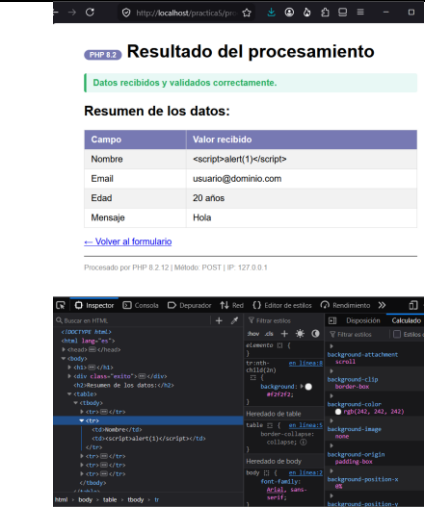
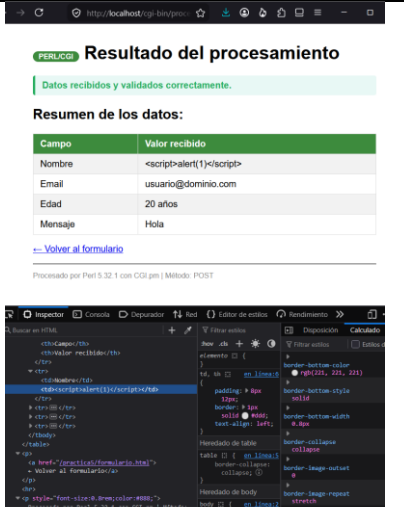
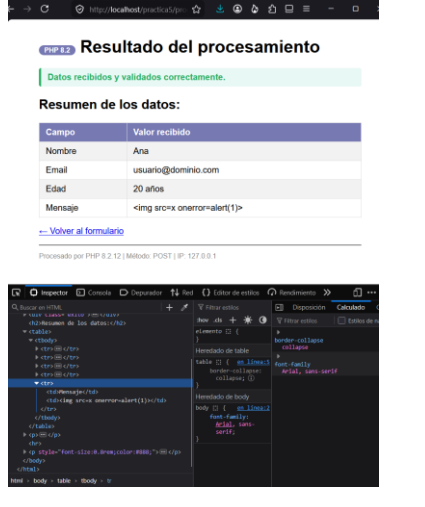
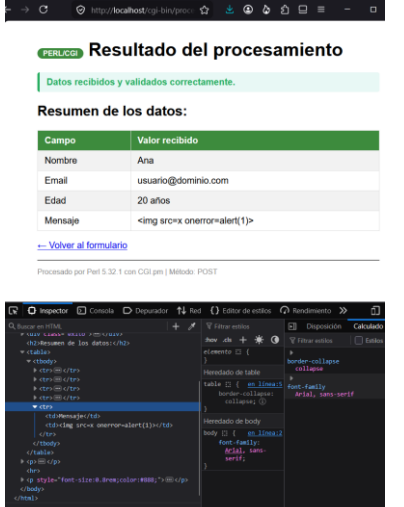
#	Caso	PHP ✓/X	PERL ✓/X
1	Campos vacíos		
2	Email sin dominio		
3	Email sin arroba		

4	Edad negativa		
5	Edad muy alta		
6	Edad no numérica		

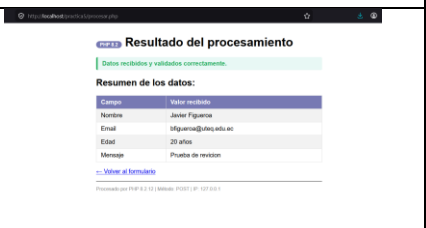
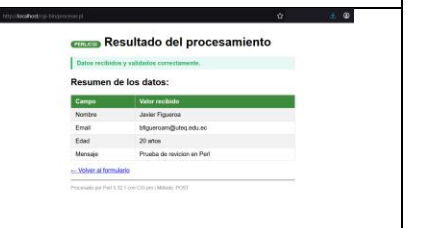
5.2 Casos de prueba 7 y 8 (XSS) — Captura + Código fuente

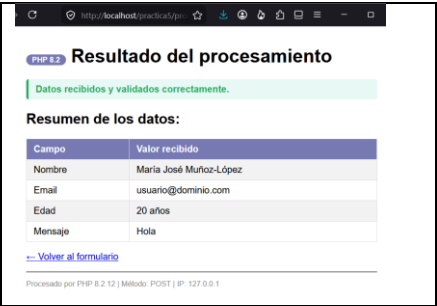
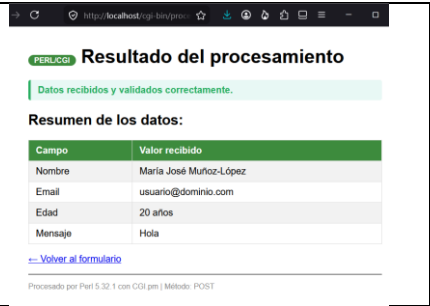
Para estos casos se debe incluir: (a) captura del resultado en pantalla con texto escapado, y (b) código fuente HTML generado (clic derecho → Ver código fuente) que confirme que los caracteres están como entidades HTML.

#	Caso	PHP ✓/X	PERL ✓/X
---	------	---------	----------

7	XSS en nombre		
8	XSS en mensaje		

5.3 Casos de prueba 9 y 10 (Datos válidos)

#	Caso	PHP ✓/X	PERL ✓/X
9	Datos válidos		

10	Nombre con caracteres especiales		
----	----------------------------------	--	---

6. Tabla Comparativa PHP vs. PERL/CGI

La siguiente tabla recoge los criterios analizados durante la práctica. La columna 'Observación personal' debe completarse con sus propias conclusiones basadas en la experiencia de implementación.

Criterio	PHP 8.2	PERL 5.38 / CGI.pm	Observación personal
Integración con Apache	Módulo nativo (mod_php)	CGI clásico: proceso nuevo por petición	Se determino que por ser modulo nativo php es mas versátil y rapido que perl
Leer datos del formulario	<code>filter_input(INPUT_POST, ...)</code>	<code>\$cgi->param('campo')</code>	Nos explicaron que protege al servidor en lugar de usar variables directas en php.
Saneamiento XSS	<code>htmlspecialchars()</code>	<code>CGI::escapeHTML()</code>	Permite que aecten caracteres especiales
Validación de email	<code>FILTER_VALIDATE_EMAIL</code>	Expresión regular manual	Se valido que contenga un formato como tal en ambos casos
Validación de entero	<code>FILTER_VALIDATE_INT</code> + rango	Regex <code>/\d+/</code> + comparación	Php realiza todo en una solo función , mientras que perl validada de manera
Cabecera HTTP	Gestionada por Apache	Obligatoria como primera salida	En perl solo responde de lamera correcta solo si se tiene el encabezado
Modo estricto	<code>declare(strict_types=1)</code>	<code>use strict; use warnings;</code>	el editor y el servidor me avisaban inmediatamente si cometía un error
Permisos de archivo	No requiere permisos especiales	<code>chmod 755</code> obligatorio en Linux/Mac	No hubo mayor problema, solo la ubicación de carpetas
Curva de aprendizaje	Media – sintaxis intuitiva	Alta – sintaxis densa y expresiva	El mas complicado fue perl porque es muy estricto
Casos de uso actuales	Desarrollo web, CMS, APIs REST	Admin. sistemas, bioinformática, scripting	Perl fue interesante pero php es más fácil

7. Reflexión Final

Responda las siguientes preguntas con un mínimo de 5 líneas en total, basándose en su experiencia personal durante la práctica:

¿Qué diferencia le pareció más significativa entre PHP y PERL?

A nivel personal, la diferencia más significativa que experimenté en el laboratorio fue la sintaxis y la facilidad para desplegar el código. Mientras que PHP 8.2 se sintió muy amigable, intuitivo y moderno al integrarse directamente dentro del entorno de Apache de forma nativa, Perl 5.38 requirió un esfuerzo mental mucho mayor. Con Perl/CGI tuve que lidiar de forma estricta con conceptos como el *shebang*, los permisos de ejecución del archivo y la impresión manual y obligatoria de la cabecera HTTP; cosas que PHP gestiona automáticamente facilitando mucho el trabajo a quienes estamos empezando en el mundo del backend.

¿Bajo qué condiciones elegiría uno u otro para un proyecto real?

Elegiría PHP 8.2 para cualquier proyecto de desarrollo web convencional, aplicaciones interactivas, sistemas CRUD rápidos o cuando trabaje en equipo, ya que su curva de aprendizaje es más accesible y cuenta con filtros nativos de seguridad excelentes. Por otro lado, elegiría Perl 5.38 únicamente bajo condiciones muy específicas donde el núcleo del proyecto no sea una interfaz web, sino tareas pesadas de backend como la administración de sistemas en el servidor, automatización de scripts de texto, análisis de datos o bioinformática, áreas donde su potente manejo de expresiones regulares sigue siendo un estándar.